

Documentación

API

ÍNDICE:

- UserController
- EmpresaController
- InstructorController
- AlumnoController
- EntregaCuadernoController
- NotaCuadernoController
- AlumnoEntregaController
- NotasEmpresaController
- GradoController
- EstanciaController
- EstanciaCompetenciaController
- NotasCompetenciaController
- SeguimientoController
- AsignaturaController
- RaController
- CompetenciaController
- CompRaController
- TransversalController
- TutorController
- AlumnoImportController

UserController

1. Inicio de sesión de usuario

- **Ruta:** POST `/api/login`
 - **Controlador:** UserController@login
 - **Descripción:** Permite iniciar sesión con correo y contraseña. Genera un token de autenticación y devuelve información del usuario, incluyendo si es tutor.
 - **Campos del Request:**
 - `email` (Obligatorio) Tipo string, formato email válido.
 - `password` (Obligatorio) Tipo string.
 - **Respuestas:**
 - `200`: Credenciales correctas, devuelve JSON con datos del usuario y token.
 - `401`: Credenciales incorrectas, devuelve JSON de error.
-

2. Cierre de sesión

- **Ruta:** POST `/api/logout`
 - **Controlador:** UserController@logout
 - **Descripción:** Cierra sesión y elimina los tokens del usuario autenticado.
 - **Respuestas:**
 - `200`: Devuelve JSON confirmando que la sesión se cerró correctamente.
-

3. Obtener usuario autenticado

- **Ruta:** GET `/api/auth`
- **Controlador:** UserController@auth
- **Descripción:** Devuelve los datos del usuario actualmente autenticado e indica si es tutor.
- **Respuestas:**
 - `200`: Devuelve JSON con la información del usuario autenticado.
 - `401`: Devuelve JSON indicando que no está autenticado.

4. Obtener listado de usuarios

- **Ruta:** GET `/api/users`
 - **Controlador:** UserController@getUsers
 - **Descripción:** Devuelve usuarios paginados. Permite filtrar por tipo, grado (si alumno) y búsqueda por texto. Incluye relaciones según tipo.
 - **Parámetros de Query:**
 - `per_page` (Opcional, default 5)
 - `tipo` (Opcional) → alumno, tutor, instructor, admin
 - `id_grado` (Opcional) → filtra alumnos por grado
 - `search` (Opcional) → búsqueda por nombre, apellidos o email
 - **Respuestas:**
 - `200`: Devuelve un JSON con la lista de usuarios paginada.
-

5. Crear usuario

- **Ruta:** POST `/api/user/create`
- **Controlador:** UserController@create
- **Descripción:** Crea un usuario (alumno, tutor, instructor, admin). En caso de alumno, se asigna grado.
- **Campos del Request:**
 - `nombre` (Obligatorio) Tipo string
 - `apellidos` (Opcional) Tipo string
 - `email` (Obligatorio) Formato email único
 - `n_tel` (Opcional) Tipo string, 9 dígitos
 - `password` (Obligatorio) Tipo string, mínimo 6 caracteres
 - `tipo` (Obligatorio) alumno, tutor, instructor, admin
 - `id_grado` (Opcional) Requerido si tipo = alumno
- **Respuestas:**
 - `201`: Usuario creado, devuelve JSON con la información del usuario.
 - `400`: Error de validación, devuelve JSON con detalles del error.

6. Actualizar usuario

- **Ruta:** PUT `/api/users/{id}`
 - **Controlador:** UserController@update
 - **Descripción:** Permite actualizar datos del usuario, incluyendo grado si es alumno.
 - **Campos del Request:**
 - `nombre` (Obligatorio) Tipo string
 - `apellidos` (Opcional) Tipo string
 - `email` (Obligatorio) Formato email, único
 - `password` (Opcional) Tipo string, mínimo 6 caracteres
 - `tipo` (Opcional) alumno, tutor, instructor, admin
 - `id_grado` (Opcional) Requerido si tipo = alumno
 - **Respuestas:**
 - `200`: Usuario actualizado, devuelve JSON con información actualizada.
 - `400`: Error de validación, devuelve JSON con detalles del error.
-

7. Cambiar contraseña

- **Ruta:** POST `/api/change-password`
- **Controlador:** UserController@changePassword
- **Descripción:** Permite cambiar la contraseña actual validando coincidencia y longitud mínima.
- **Campos del Request:**
 - `current_password` (Obligatorio)
 - `new_password` (Obligatorio) mínimo 6 caracteres
 - `new_password_confirmation` (Obligatorio) debe coincidir con `new_password`
- **Respuestas:**
 - `200`: Contraseña actualizada correctamente, devuelve JSON confirmando.
 - `400`: Contraseña actual incorrecta, devuelve JSON con error.

8. Borrar usuario

- **Ruta:** DELETE `/api/users/{id}`
- **Controlador:** UserController@delete
- **Descripción:** Elimina un usuario. Si es instructor, desasigna a sus alumnos antes de borrar.
- **Respuestas:**
 - **200:** Usuario eliminado, devuelve JSON confirmando.
 - **404:** Usuario no encontrado, devuelve JSON con error.

EmpresaController

1. Obtener empresas paginadas con búsqueda

- **Ruta:** GET `/api/empresas`
 - **Controlador:** EmpresaController@getCompanyys
 - **Descripción:** Devuelve un listado paginado de empresas. Permite filtrar mediante búsqueda por CIF, nombre, dirección, email o teléfono.
 - **Parámetros de Query (Opcional):**
 - `q`: busca filtrando coincidencias parciales en CIF, Nombre, Dirección, Email o N_Tel
 - **Paginación:** fija, 5 resultados por página
 - **Respuestas:**
 - `200`: Devuelve JSON con listado paginado de empresas.
-

2. Obtener todas las empresas (sin paginación)

- **Ruta:** GET `/api/allempresas`
 - **Controlador:** EmpresaController@getAllCompanyys
 - **Descripción:** Devuelve las empresas registradas en la bbdd en un JSON.
 - **Respuestas:**
 - `200`: Devuelve JSON con todas las empresas.
-

3. Crear una nueva empresa

- **Ruta:** POST `/api/empresa/create`
- **Controlador:** EmpresaController@create
- **Descripción:** Permite registrar una nueva empresa validando.
- **Campos del Request:**
 - `Nombre` (Obligatorio) Tipo string, máximo 255 caracteres
 - `Direccion` (Opcional) Tipo string, máximo 255 caracteres
 - `CIF` (Obligatorio) Tipo string, máximo 20 caracteres, único
 - `Email` (Opcional) Tipo string, formato email, único
 - `N_Tel` (Opcional) Tipo string, exactamente 9 dígitos, único
- **Respuestas:**
 - `201`: Empresa creada, devuelve JSON con los datos de la empresa.
 - `400`: Error de validación, devuelve JSON con los campos que fallaron.

InstructorController

1. Obtener instructores de una empresa

- **Ruta:** GET `/api/empresa/{cif}/instructores`
 - **Controlador:** InstructorController@getCompanyInstructor
 - **Descripción:** Devuelve la lista de instructores asociados a una empresa específica mediante su CIF. Incluye los datos del usuario relacionado.
 - **Parámetros de URL:**
 - `cif` (Obligatorio): CIF de la empresa
 - **Respuestas:**
 - `200`: Devuelve JSON con la lista de instructores.
 - `404`: Si no se encuentra la empresa o instructores, devuelve JSON con mensaje de error.
-

2. Crear un nuevo instructor

- **Ruta:** POST `/api/empresa/instructor/create`
- **Controlador:** InstructorController@crearInstructor
- **Descripción:** Crea un usuario de tipo instructor y lo vincula a una empresa mediante su CIF.
- **Campos del Request:**
 - `nombre` (Obligatorio) Tipo string, máximo 255 caracteres
 - `apellidos` (Opcional) Tipo string, máximo 255 caracteres
 - `email` (Obligatorio) Tipo string, formato email, único
 - `n_tel` (Opcional) Tipo string, exactamente 9 dígitos, único
 - `password` (Obligatorio) Tipo string, mínimo 6 caracteres
 - `CIF_Empresa` (Obligatorio) Tipo string, debe existir en tabla empresa
- **Respuestas:**
 - `201`: Instructor creado, devuelve JSON con los datos del instructor.
 - `400`: Error de validación, devuelve JSON con los campos que fallaron.

AlumnoController

1. Obtener alumnos de un tutor

- **Ruta:** GET `/api/tutores/{id}/alumnos`
 - **Controlador:** AlumnoController@alumnosDeTutor
 - **Descripción:** Devuelve todos los alumnos asociados a un tutor específico. Solo accesible para admin o el tutor correspondiente. Permite búsqueda por nombre, apellidos o email.
 - **Parámetros de URL:**
 - `id` (Obligatorio): ID del tutor
 - **Parámetros de Query (opcional):**
 - `q` → Texto para filtrar alumnos por nombre, apellidos o email
 - `per_page` → Resultados por página (default 5)
 - **Respuestas:**
 - `200`: Devuelve JSON con la lista de alumnos.
 - `403`: No autorizado, devuelve JSON con mensaje de error.
-

2. Obtener alumnos de un instructor

- **Ruta:** GET `/api/instructores/{id}/alumnos`
- **Controlador:** AlumnoController@alumnosDelInstructor
- **Descripción:** Devuelve todos los alumnos asociados a un instructor específico. Solo accesible para admin o el instructor correspondiente.
- **Parámetros de URL:**
 - `id` (Obligatorio): ID del instructor
- **Respuestas:**
 - `200`: Devuelve JSON con la lista de alumnos.
 - `403`: No autorizado, devuelve JSON con mensaje de error.

3. Obtener grado de un alumno

- **Ruta:** GET `/api/alumno/{id}`
 - **Controlador:** AlumnoController@getGrado
 - **Descripción:** Devuelve el grado asociado a un alumno específico.
 - **Parámetros de URL:**
 - `id` (Obligatorio): ID del alumno
 - **Respuestas:**
 - `200`: Devuelve JSON con el grado del alumno.
 - `404`: Alumno no encontrado, devuelve JSON con mensaje de error.
-

4. Obtener todas las notas de un alumno

- **Ruta:** GET `/api/alumno/{id}/mis-notasAlumno`
 - **Controlador:** AlumnoController@misNotasAlumno
 - **Descripción:** Devuelve todas las notas de un alumno, incluyendo cuaderno, competencias técnicas, transversales, empresa y notas Egibide. Calcula las notas finales por asignatura.
 - **Parámetros de URL:**
 - `id` (Obligatorio): ID del alumno
 - **Respuestas:**
 - `200`: Devuelve JSON con todas las notas calculadas.
-

5. Obtener notas generales de un alumno

- **Ruta:** GET `/api/alumno/{id}/mis-notas`
- **Controlador:** AlumnoController@misNotas
- **Descripción:** Devuelve todas las notas y relaciones del alumno incluyendo cuadernos, competencias, transversales y notas Egibide.
- **Parámetros de URL:**
 - `id` (Obligatorio): ID del alumno
- **Respuestas:**
 - `200`: Devuelve JSON con todas las notas.
 - `200` (ID nulo): Devuelve JSON vacío.

6. Guardar nota Egibide

- **Ruta:** POST `/api/alumnos/{idAlumno}/nota-egibide`
 - **Controlador:** AlumnoController@guardarNotaEgibide
 - **Descripción:** Guarda o actualiza la nota Egibide de un alumno para una asignatura específica. Solo admin o tutor autorizado.
 - **Campos del Request:**
 - `id_asignatura` (Obligatorio) ID de la asignatura
 - `nota` (Obligatorio) Valor numérico de la nota
 - **Respuestas:**
 - `200`: Nota guardada correctamente, devuelve JSON con los datos de la nota.
 - `403`: No autorizado, devuelve JSON con mensaje de error.
 - `400`: Error de validación, devuelve JSON con los campos que fallaron.
-

7. Asignar instructor a un alumno

- **Ruta:** PUT `/api/alumnos/{idAlumno}/asignar-instructor`
- **Controlador:** AlumnoController@asignarInstructor
- **Descripción:** Permite a un tutor o admin asignar un instructor a un alumno.
- **Campos del Request:**
 - `ID_Instructor` (Obligatorio) ID del instructor
- **Respuestas:**
 - `200`: Instructor asignado correctamente, devuelve JSON con los datos del alumno y el instructor asignado.
 - `403`: No autorizado, devuelve JSON con mensaje de error.
 - `400`: Error de validación, devuelve JSON con los campos que fallaron.

EntregaCuadernoController

1. Crear entrega de cuaderno para un grado

- **Ruta:** POST `/api/grado/{gradoId}/entregas`
 - **Controlador:** EntregaCuadernoController@store
 - **Descripción:** Crea una nueva entrega de cuaderno asociada a un grado específico.
 - **Parámetros de URL:**
 - `gradoId` (Obligatorio): ID del grado
 - **Campos del Request:**
 - `Descripcion` (Obligatorio) Tipo string
 - `Fecha_Limite` (Obligatorio) Fecha límite de entrega
 - **Respuestas:**
 - `200`: Entrega creada, devuelve JSON con los datos de la entrega.
 - `400`: Error de validación, devuelve JSON con los campos que fallaron.
-

2. Obtener entregas de un grado

- **Ruta:** GET `/api/grado/{gradoId}/entregas`
- **Controlador:** EntregaCuadernoController@porGrado
- **Descripción:** Devuelve todas las entregas asociadas a un grado, incluyendo la entrega de los alumnos filtrada por tutor (opcional).
- **Parámetros de URL:**
 - `gradoId` (Obligatorio): ID del grado
- **Parámetros de Query (opcional):**
 - `tutor_id` → ID del tutor para filtrar alumnos
- **Respuestas:**
 - `200`: Devuelve JSON con las entregas del grado y las entregas de los alumnos asociadas.

3. Obtener entregas de un alumno

- **Ruta:** GET `/api/entregas/alumno/{id}`
 - **Controlador:** EntregaCuadernoController@entregasAlumno
 - **Descripción:** Devuelve todas las entregas disponibles para el grado del alumno y su entrega si ya existe.
 - **Parámetros de URL:**
 - `id` (Obligatorio): ID del alumno (usuario)
 - **Respuestas:**
 - `200`: Devuelve JSON con las entregas del alumno.
 - `403`: No autorizado si el ID del usuario no coincide con el alumno autenticado, devuelve JSON con mensaje de error.
-

4. Eliminar entrega

- **Ruta:** DELETE `/api/grado/{gradoId}/entregas/{entregaId}`
- **Controlador:** EntregaCuadernoController@destroy
- **Descripción:** Elimina una entrega específica de un grado.
- **Parámetros de URL:**
 - `gradoId` (Obligatorio): ID del grado
 - `entregaId` (Obligatorio): ID de la entrega
- **Respuestas:**
 - `200`: Entrega eliminada correctamente, devuelve JSON con mensaje de éxito.
 - `404`: Entrega no encontrada, devuelve JSON con mensaje de error.

NotaCuadernoController

1. Guardar observaciones y feedback de un alumno

- **Ruta:** POST `/api/observacionesCuadernoAlumno`
 - **Controlador:** `NotaCuadernoController@observacionesCuadernoAlumno`
 - **Descripción:** Permite crear o actualizar observaciones y feedback de la entrega de un alumno.
 - **Campos del Request:**
 - `ID_Cuaderno` (Obligatorio) → ID del cuaderno
 - `Observaciones` (Opcional) → Tipo string
 - `Feedback` (Opcional) → Tipo string
 - **Respuestas:**
 - `200`: Observación creada correctamente, devuelve JSON con los datos de la observación.
 - `400`: Validación fallida, devuelve JSON con los campos que no cumplen los requisitos.
-

2. Guardar nota de cuaderno de un alumno

- **Ruta:** POST `/api/nota-cuaderno`
- **Controlador:** `NotaCuadernoController@notaCuaderno`
- **Descripción:** Crea o actualiza la nota de un cuaderno para un alumno.
- **Campos del Request:**
 - `ID_Alumno` (Obligatorio) → ID del alumno
 - `Nota` (Obligatorio) → Valor numérico de la nota
- **Respuestas:**
 - `200`: Nota creada correctamente, devuelve JSON con los datos de la nota.
 - `400`: Validación fallida, devuelve JSON con los campos que no cumplen los requisitos.

3. Obtener notas de cuaderno por tutor

- **Ruta:** GET `/api/tutor/{tutorId}/notas-cuaderno`
- **Controlador:** NotaCuadernoController@notasPorTutor
- **Descripción:** Devuelve todos los alumnos asociados a un tutor junto con su nota de cuaderno.
- **Parámetros de URL:**
 - `tutorId` (Obligatorio) → ID del tutor
- **Respuestas:**
 - `200`: Devuelve JSON con el listado de alumnos y sus notas de cuaderno.

AlumnoEntregaController

1. Entregar cuaderno de un alumno

- **Ruta:** POST `/api/alumno/{id}/entrega-cuaderno`
 - **Controlador:** AlumnoEntregaController@entregarCuaderno
 - **Descripción:** Permite a un alumno subir su cuaderno para una entrega específica. Si ya existe una entrega, se actualiza.
 - **Parámetros de URL:**
 - `id` (Obligatorio) → ID del alumno (usuario)
 - **Campos del Request (multipart/form-data):**
 - `cuaderno` (Obligatorio) → Archivo PDF o Word del cuaderno
 - `ID_Entrega` (Obligatorio) → ID de la entrega (EntregaCuaderno)
 - **Respuestas:**
 - `200`: Cuaderno subido correctamente, devuelve JSON con información del alumno, la entrega y URL del archivo.
 - `400`: Error de validación, devuelve JSON con los campos que no cumplen los requisitos.
 - `403`: No autorizado, el alumno intenta subir un cuaderno de otro usuario, devuelve JSON de error.
-

2. Descargar cuaderno de alumno

- **Ruta:** GET `/api/alumno/entrega-cuaderno/{id}/descargar`
- **Controlador:** AlumnoEntregaController@descargarCuaderno
- **Descripción:** Permite descargar el archivo subido de una entrega específica.
- **Parámetros de URL:**
 - `id` (Obligatorio) → ID de AlumnoEntrega
- **Respuestas:**
 - `200`: Descarga del archivo correctamente.
 - `404`: Archivo no encontrado, devuelve JSON con mensaje de error.

NotasEmpresaController

1. Ver competencias y notas de un alumno

- **Ruta:** GET `/api/alumnos/{idAlumno}/notas`
 - **Controlador:** NotasEmpresaController@show
 - **Descripción:** Devuelve todas las competencias técnicas y transversales de un alumno, incluyendo las notas actuales.
 - **Parámetros de URL:**
 - `idAlumno` (Obligatorio) → ID del alumno
 - **Seguridad:** Solo accesible por admin, tutor del alumno, o instructor asignado.
 - **Respuestas:**
 - `200`: Devuelve un JSON con las competencias transversales y técnicas del alumno, incluyendo las notas.
 - `403`: No autorizado, devuelve JSON con mensaje de error.
-

2. Guardar o actualizar notas de un alumno

- **Ruta:** POST `/api/alumnos/{idAlumno}/notas`
- **Controlador:** NotasEmpresaController@store
- **Descripción:** Permite guardar o actualizar las notas de competencias transversales y técnicas de un alumno. Si se omite un valor, la nota no se modifica.
- **Parámetros de URL:**
 - `idAlumno` (Obligatorio) → ID del alumno
- **Campos del Request:**
 - `transversales` (Opcional) → JSON con ID de la competencia transversal y nota correspondiente.
 - `tecnicas` (Opcional) → JSON con ID de la competencia técnica y nota correspondiente.
- **Respuestas:**
 - `200`: Notas guardadas correctamente, devuelve JSON confirmando la operación.
 - `403`: No autorizado, si no es admin, tutor o instructor, devuelve JSON con mensaje de error.

GradoController

1. Obtener grados con paginación y búsqueda

- **Ruta:** GET /api/grados
 - **Controlador:** GradoController@getGrados
 - **Descripción:** Devuelve un listado paginado de grados. Permite filtrar mediante búsqueda por nombre o curso.
 - **Parámetros de Query (opcional):**
 - q → texto de búsqueda por nombre o curso
 - per_page → cantidad de resultados por página (default 5)
 - **Respuestas:**
 - 200: Devuelve un JSON con los grados paginados, incluyendo nombre, curso e ID de tutor.
-

2. Obtener todos los grados (sin paginación)

- **Ruta:** GET /api/gradosTodos
 - **Controlador:** GradoController@getTodosGrados
 - **Descripción:** Devuelve todos los grados registrados en la base de datos sin paginación.
 - **Respuestas:**
 - 200: Devuelve un JSON con todos los grados.
-

3. Crear un nuevo grado

- **Ruta:** POST /api/grados
- **Controlador:** GradoController@crearGrado
- **Descripción:** Permite crear un nuevo grado y asignarle un tutor opcional. Valida que el tutor no tenga ya un grado asignado.
- **Campos del Request:**
 - nombre (Obligatorio) → nombre del grado
 - curso (Opcional) → curso del grado
 - id_tutor (Opcional) → ID del tutor que se asignará al grado
- **Respuestas:**
 - 201: Grado creado correctamente, devuelve información del grado creado.
 - 422: Error, el tutor ya tiene un grado asignado.
 - 400: Error de validación, campos requeridos o longitud máxima incumplida.

4. Eliminar un grado

- **Ruta:** `DELETE /api/grados/{id}`
 - **Controlador:** `GradoController@eliminarGrado`
 - **Descripción:** Elimina un grado si no tiene alumnos asignados.
 - **Parámetros de URL:**
 - `id` (Obligatorio) → ID del grado
 - **Respuestas:**
 - **200:** Grado eliminado correctamente.
 - **422:** No se puede eliminar el grado porque tiene alumnos asignados.
 - **404:** Grado no encontrado.
-

5. Obtener asignaturas de un grado

- **Ruta:** `GET /api/grados/{id}/asignaturas`
 - **Controlador:** `GradoController@getAsignaturas`
 - **Descripción:** Devuelve las asignaturas asociadas a un grado específico
 - **Parámetros de URL:**
 - `id` (Obligatorio) → ID del grado
 - **Respuestas:**
 - **200:** Devuelve un JSON con todas las asignaturas del grado.
 - **404:** Grado no encontrado.
-

6. Obtener competencias de un grado

- **Ruta:** `GET /api/grados/{id}/competencias`
- **Controlador:** `GradoController@getCompetencias`
- **Descripción:** Devuelve todas las competencias asociadas a un grado, incluyendo relaciones a asignaturas y RAs.
- **Parámetros de URL:**
 - `id` (Obligatorio) → ID del grado
- **Respuestas:**
 - **200:** Devuelve un JSON con todas las competencias del grado.
 - **404:** Grado no encontrado.

7. Gestión de tutor para su grado

- **Ruta:** GET /api/mi-grado/gestion
- **Controlador:** GradoController@getDatosGestionTutor
- **Descripción:** Devuelve información detallada para el tutor autenticado: grado asignado, asignaturas, alumnos matriculados y notas calculadas por alumno.
- **Respuestas:**
 - **200:** Devuelve un JSON con el grado, las asignaturas y los alumnos con sus notas calculadas.
 - **404:** Sin grado asignado, devuelve mensaje de error.

EstanciaController

1. Obtener todas las estancias

- **Ruta:** GET /api/estancias
 - **Controlador:** EstanciaController@getEstancias
 - **Descripción:** Devuelve todas las estancias registradas en el sistema.
 - **Respuestas:**
 - **200:** Devuelve un JSON con todas las estancias.
-

2. Historial de estancias de un alumno (para tutor)

- **Ruta:** GET /api/tutor/alumno/{id}/estancias
 - **Controlador:** EstanciaController@historialEstanciasAlumno
 - **Descripción:** Devuelve el historial completo de estancias de un alumno, incluyendo empresa, horario, tutor, instructor y grado.
 - **Parámetros de URL:**
 - **id** (Obligatorio) → ID del alumno
 - **Respuestas:**
 - **200:** Devuelve un JSON con todas las estancias del alumno, ordenadas por fecha de inicio descendente.
 - **404:** Alumno o estancias no encontradas.
-

3. Obtener estancia actual de un alumno (para alumno)

- **Ruta:** GET /api/alumno/{id}/estancia
 - **Controlador:** EstanciaController@getEstanciaActual
 - **Descripción:** Devuelve únicamente la estancia actual del alumno, incluyendo empresa, horario, tutor e instructor.
 - **Parámetros de URL:**
 - **id** (Obligatorio) → ID del alumno
 - **Respuestas:**
 - **200:** Devuelve un JSON con la estancia actual del alumno.
 - **404:** Alumno o estancia no encontrada.
-

4. Obtener alumnos de una empresa

- **Ruta:** `GET /api/empresa/{cif}/alumnos`
 - **Controlador:** `EstanciaController@getCompanyAlumnos`
 - **Descripción:** Devuelve todos los alumnos asociados a una empresa específica mediante su CIF. Incluye información del alumno e instructor asignado.
 - **Parámetros de URL:**
 - `cif` (Obligatorio) → CIF de la empresa
 - **Respuestas:**
 - **200:** Devuelve un JSON con los alumnos vinculados a la empresa.
 - **404:** Empresa no encontrada.
-

5. Asignar una estancia a un alumno

- **Ruta:** `POST /api/asignarEstancia`
- **Controlador:** `EstanciaController@asignarEstancia`
- **Descripción:** Permite asignar una estancia a un alumno, con fechas, horarios y un instructor opcional. Los horarios se agrupan automáticamente si son iguales.
- **Campos del Request:**
 - `ID_Alumno` (Obligatorio) → ID del alumno
 - `CIF_Empresa` (Obligatorio) → CIF de la empresa
 - `Fecha_inicio` (Obligatorio) → Fecha de inicio de la estancia
 - `Fecha_fin` (Obligatorio) → Fecha de fin de la estancia
 - `ID_Instructor` (Opcional) → ID del instructor asignado
 - `horarios` (Obligatorio) → Array de horarios
 - `Dia` (Obligatorio) → Día de la semana
 - `Horario1` (Obligatorio) → Primer turno
 - `Horario2` (Opcional) → Segundo turno
- **Respuestas:**
 - **201:** Estancia creada correctamente, devuelve la estancia con los horarios asociados.
 - **400:** Error de validación de los campos requeridos.

6. Obtener competencias de una estancia

- **Ruta:** GET /api/estancias/{id}/competencias
 - **Controlador:** EstanciaController@competencias
 - **Descripción:** Devuelve todas las competencias asociadas a una estancia, incluyendo las notas del alumno específico.
 - **Parámetros de URL:**
 - **id** (Obligatorio) → ID de la estancia
 - **ID_Alumno** (Obligatorio, Query o Request) → ID del alumno para filtrar sus notas
 - **Respuestas:**
 - **200:** Devuelve un JSON con las competencias y las notas del alumno.
 - **404:** Estancia o competencias no encontradas.
-

7. Eliminar una estancia

- **Ruta:** DELETE /api/estancia/{id}
- **Controlador:** EstanciaController@eliminarEstancia
- **Descripción:** Elimina una estancia específica. Solo accesible para admin o tutor. Además, elimina la relación del alumno con el instructor.
- **Parámetros de URL:**
 - **id** (Obligatorio) → ID de la estancia
- **Respuestas:**
 - **200:** Estancia eliminada correctamente.
 - **403:** No autorizado si el usuario no es admin o tutor.
 - **404:** Estancia no encontrada.

EstanciaCompetenciaController

1. Añadir competencia a una estancia

- **Ruta:** `POST /api/estancias/{estanciaId}/competencias`
 - **Controlador:** `EstanciaCompetenciaController@create`
 - **Descripción:** Permite asociar una competencia existente a una estancia específica.
 - **Parámetros de URL:**
 - `estanciaId` (Obligatorio) → ID de la estancia
 - **Campos del Request:**
 - `ID_Comp` (Obligatorio) → ID de la competencia a asignar
 - **Respuestas:**
 - **201:** Competencia añadida correctamente, devuelve JSON confirmando la operación.
 - **409:** La competencia ya está asignada a la estancia, devuelve JSON con mensaje de conflicto.
 - **400:** Error de validación si el ID de la competencia no existe.
-

2. Eliminar competencia de una estancia

- **Ruta:** `DELETE /api/estancias/{estanciaId}/competencias/{competenciaId}`
- **Controlador:** `EstanciaCompetenciaController@delete`
- **Descripción:** Elimina una competencia previamente asignada a una estancia. También elimina la nota asociada a esa competencia para el alumno correspondiente.
- **Parámetros de URL:**
 - `estanciaId` (Obligatorio) → ID de la estancia
 - `competenciaId` (Obligatorio) → ID de la competencia
- **Respuestas:**
 - **200:** Competencia eliminada correctamente de la estancia, devuelve JSON confirmando la operación.
 - **404:** Competencia no asignada a esa estancia o estancia no encontrada.

NotasCompetenciaController

1. Guardar o actualizar nota de una competencia

- **Ruta:** POST
/api/alumnos/{alumnoId}/competencias/{competenciaId}/nota
- **Controlador:** NotasCompetenciaController@guardarNota
- **Descripción:** Permite guardar o actualizar la nota de una competencia específica para un alumno. Si la nota ya existe, se actualiza; si no, se crea.
- **Parámetros de URL:**
 - **alumnoId** (Obligatorio) → ID del alumno
 - **competenciaId** (Obligatorio) → ID de la competencia
- **Campos del Request:**
 - **nota** (Obligatorio) → Número entero entre 1 y 4
- **Validaciones:**
 - **nota** → Obligatorio, entero, valor permitido entre 1 y 4
- **Respuestas:**
 - **200:** Nota guardada correctamente, devuelve JSON confirmando la operación.
 - **400:** Error de validación si la nota no cumple los requisitos establecidos.

SeguimientoController

1. Obtener seguimientos de una estancia

- **Ruta:** GET /api/estancias/{idEstancia}/seguimientos
 - **Controlador:** SeguimientoController@index
 - **Descripción:** Devuelve todos los registros de seguimiento asociados a una estancia concreta.
 - **Parámetros de URL:**
 - **idEstancia** (Obligatorio) → ID de la estancia
 - **Respuestas:**
 - **200:** Devuelve un JSON con la lista de seguimientos asociados a la estancia.
 - **200:** Si la estancia no existe, devuelve un JSON vacío.
-

2. Crear nuevo seguimiento

- **Ruta:** POST /api/seguimientos
- **Controlador:** SeguimientoController@crearSeguimiento
- **Descripción:** Permite crear un nuevo registro de seguimiento asociado a una estancia.
- **Campos del Request:**
 - **ID_Estancia** (Obligatorio) → ID de la estancia existente.
 - **Fecha** (Obligatorio) → Fecha del seguimiento (formato fecha válido).
 - **Hora** (Obligatorio) → Hora del seguimiento.
 - **Accion_seguimiento** (Obligatorio) → Descripción de la acción realizada.
 - **Seguimiento_actividad** (Opcional) → Detalle adicional de la actividad.
- **Validaciones:**
 - **ID_Estancia** debe existir en la tabla **Estancia_alumno**.
 - **Fecha** debe ser una fecha válida.
 - **Accion_seguimiento** obligatorio y tipo string.
- **Respuestas:**
 - **201:** Seguimiento creado correctamente, devuelve un JSON con los datos del seguimiento creado.
 - **400:** Error de validación si los datos no cumplen los requisitos.

3. Modificar seguimiento

- **Ruta:** `PUT /api/seguimientos/{id}`
 - **Controlador:** `SeguimientoController@ModificarSeguimiento`
 - **Descripción:** Permite actualizar los datos de un seguimiento existente.
 - **Parámetros de URL:**
 - `id` (Obligatorio) → ID del seguimiento
 - **Campos del Request:**
 - `Fecha` (Obligatorio) → Fecha actualizada.
 - `Hora` (Obligatorio) → Hora actualizada.
 - `Accion_seguimiento` (Obligatorio) → Nueva descripción de la acción.
 - `Seguimiento_actividad` (Opcional) → Detalle adicional actualizado.
 - **Respuestas:**
 - **200:** Seguimiento actualizado correctamente, devuelve un JSON con los datos actualizados.
 - **404:** Seguimiento no encontrado.
 - **400:** Error de validación.
-

4. Eliminar seguimiento

- **Ruta:** `DELETE /api/seguimientos/{id}`
- **Controlador:** `SeguimientoController@eliminarSeguimiento`
- **Descripción:** Elimina un registro de seguimiento existente.
- **Parámetros de URL:**
 - `id` (Obligatorio) → ID del seguimiento
- **Respuestas:**
 - **200:** Seguimiento eliminado correctamente, devuelve JSON confirmando la operación.
 - **404:** Seguimiento no encontrado.

AsignaturaController

1. Obtener RAs de una asignatura

- **Ruta:** `GET /api/asignaturas/{id}/ras`
 - **Controlador:** `AsignaturaController@getRas`
 - **Descripción:** Devuelve todos los Resultados de Aprendizaje (RAs) asociados a una asignatura concreta.
 - **Parámetros de URL:**
 - `id` (Obligatorio) → ID de la asignatura
 - **Respuestas:**
 - **200:** Devuelve un JSON con la lista de RAs asociados a la asignatura.
 - **200:** Si la asignatura no tiene RAs, devuelve un array vacío.
-

2. Crear asignatura

- **Ruta:** `POST /api/asignaturas`
- **Controlador:** `AsignaturaController@store`
- **Descripción:** Permite crear una nueva asignatura asociada a un grado específico.
- **Campos del Request:**
 - `nombre` (Obligatorio) → Nombre de la asignatura.
 - `ID_Grado` (Obligatorio) → ID del grado al que pertenece la asignatura.
- **Validaciones:**
 - `nombre` obligatorio, tipo string y máximo 255 caracteres.
 - `ID_Grado` obligatorio y debe existir en la tabla `grado`.
- **Respuestas:**
 - **201:** Asignatura creada correctamente, devuelve un JSON con los datos de la asignatura creada.
 - **400:** Error de validación si los datos no cumplen los requisitos.

3. Eliminar asignatura

- **Ruta:** `DELETE /api/asignaturas/{id}`
- **Controlador:** `AsignaturaController@destroy`
- **Descripción:** Elimina una asignatura existente. Si está configurado en la base de datos con `onDelete('cascade')`, también se eliminarán automáticamente sus RAs asociados.
- **Parámetros de URL:**
 - `id` (Obligatorio) → ID de la asignatura
- **Respuestas:**
 - **200:** Asignatura eliminada correctamente, devuelve JSON confirmando la operación.
 - **404:** Asignatura no encontrada.

RaController

1. Crear un nuevo RA (Resultado de Aprendizaje)

- **Ruta:** `POST /api/ras`
 - **Controlador:** `RaController@store`
 - **Descripción:** Permite crear un nuevo RA asociado a una asignatura específica.
 - **Campos del Request:**
 - `Descripcion` (Obligatorio) → Texto descriptivo del RA.
 - `ID_Asignatura` (Obligatorio) → ID de la asignatura a la que pertenece el RA.
 - **Validaciones:**
 - `Descripcion` obligatorio, tipo string, máximo 255 caracteres.
 - `ID_Asignatura` obligatorio y debe existir en la tabla `asignatura`.
 - **Respuestas:**
 - **201:** RA creado correctamente, devuelve un JSON con los datos del RA creado.
 - **400:** Error de validación si los datos no cumplen los requisitos.
-

2. Eliminar un RA

- **Ruta:** `DELETE /api/ras/{id}`
- **Controlador:** `RaController@destroy`
- **Descripción:** Permite eliminar un RA existente.
- **Parámetros de URL:**
 - `id` (Obligatorio) → ID del RA a eliminar
- **Respuestas:**
 - **200:** RA eliminado correctamente, devuelve JSON confirmando la operación.
 - **404:** RA no encontrado.

CompetenciaController

1. Crear una nueva competencia

- **Ruta:** `POST /api/competencias`
 - **Controlador:** `CompetenciaController@store`
 - **Descripción:** Permite crear una nueva competencia asociada a un grado específico.
 - **Campos del Request:**
 - `descripcion` (Obligatorio) → Texto descriptivo de la competencia.
 - `ID_Grado` (Obligatorio) → ID del grado al que pertenece la competencia.
 - **Validaciones:**
 - `descripcion` obligatorio, tipo string, máximo 255 caracteres.
 - `ID_Grado` obligatorio y debe existir en la tabla `grado`.
 - **Respuestas:**
 - **201:** Competencia creada correctamente, devuelve JSON con los datos de la competencia.
 - **400:** Error de validación si los datos no cumplen los requisitos.
-

2. Eliminar una competencia

- **Ruta:** `DELETE /api/competencias/{id}`
- **Controlador:** `CompetenciaController@destroy`
- **Descripción:** Permite eliminar una competencia existente.
- **Parámetros de URL:**
 - `id` (Obligatorio) → ID de la competencia a eliminar
- **Respuestas:**
 - **200:** Competencia eliminada correctamente, devuelve JSON confirmando la operación.
 - **404:** Competencia no encontrada.

CompRaController

1. Obtener competencias y RAs de un grado

- **Ruta:** GET /api/compra/{gradoId}
 - **Controlador:** CompRaController@getCompRa
 - **Descripción:** Devuelve todas las competencias de un grado junto con las asignaturas y sus RAs asociadas a esas competencias.
 - **Parámetros de URL:**
 - **gradoId** (Obligatorio) → ID del grado del que se desean obtener las competencias y RAs.
 - **Respuestas:**
 - **200:** Devuelve un JSON con:
 - **competencias** → listado de competencias del grado
 - **asignaturas** → listado de asignaturas con sus RAs y las competencias relacionadas
-

2. Crear o eliminar relación RA-Competencia

- **Ruta:** POST /api/compra
- **Controlador:** CompRaController@createOrDelete
- **Descripción:** Crea una relación entre una competencia y un RA dentro de una asignatura. Si la relación ya existe, la elimina.
- **Campos del Request:**
 - **ID_Comp** (Obligatorio) → ID de la competencia
 - **ID_Ra** (Obligatorio) → ID del resultado de aprendizaje (RA)
 - **ID_Asignatura** (Obligatorio) → ID de la asignatura
- **Respuestas:**
 - **200:** Relación creada o eliminada correctamente, devuelve JSON indicando la acción (**created** o **deleted**) y un mensaje.
 - **400:** Error de validación si faltan campos o no cumplen el tipo requerido.

TransversalController

1. Obtener transversales de un alumno

- **Ruta:** `GET /api/transversales/alumno/{idAlumno}`
 - **Controlador:** `TransversalController@getTransversalesAlumno`
 - **Descripción:** Devuelve todas las competencias transversales del sistema con la nota actual de un alumno.
 - **Parámetros de URL:**
 - `idAlumno` (Obligatorio) → ID del alumno
 - **Seguridad:** Solo accesible por admin, tutor del alumno, o instructor asignado.
 - **Respuestas:**
 - **200:** JSON con las transversales y la nota correspondiente del alumno (null si no hay nota).
 - **403:** No autorizado, devuelve JSON con mensaje de error.
-

2. Actualizar nota de transversal para un alumno

- **Ruta:** `PUT /api/alumnos/{idAlumno}/transversales/{transversalId}/nota`
- **Controlador:** `TransversalController@actualizarNotaTransversal`
- **Descripción:** Permite actualizar o eliminar la nota de una transversal para un alumno. Si se envía `null` como nota, se elimina el registro existente.
- **Parámetros de URL:**
 - `idAlumno` (Obligatorio) → ID del alumno
 - `transversalId` (Obligatorio) → ID de la competencia transversal
- **Campos del Request:**
 - `nota` (Opcional) → Valor entero entre 1 y 4.
- **Seguridad:** Solo accesible por admin, tutor del alumno, o instructor asignado.
- **Respuestas:**
 - **200:** JSON confirmando que la nota ha sido guardada o eliminada correctamente.
 - **403:** No autorizado, devuelve JSON con mensaje de error.

3. Obtener todas las transversales

- **Ruta:** GET /api/transversales
 - **Controlador:** TransversalController@getTransversales
 - **Descripción:** Devuelve todas las competencias transversales del sistema.
 - **Respuestas:**
 - **200:** JSON con todas las transversales existentes.
-

4. Crear nueva transversal

- **Ruta:** POST /api/transversales
 - **Controlador:** TransversalController@crearTransversal
 - **Descripción:** Permite crear una nueva competencia transversal.
 - **Campos del Request:**
 - **descripcion** (Obligatorio) → Descripción de la transversal, máximo 255 caracteres
 - **Respuestas:**
 - **201:** JSON confirmando la creación de la transversal y mostrando la nueva transversal.
 - **422:** Error de validación, devuelve JSON con mensaje de error.
-

5. Actualizar transversal

- **Ruta:** PUT /api/transversales/{id}
- **Controlador:** TransversalController@actualizarTransversal
- **Descripción:** Permite actualizar la descripción de una transversal existente.
- **Parámetros de URL:**
 - **id** (Obligatorio) → ID de la transversal
- **Campos del Request:**
 - **descripcion** (Obligatorio) → Nueva descripción, máximo 255 caracteres
- **Respuestas:**
 - **200:** JSON confirmando la actualización y mostrando la transversal actualizada.
 - **404:** Transversal no encontrada.
 - **422:** Error de validación, devuelve JSON con mensaje de error.

6. Eliminar transversal

- **Ruta:** `DELETE /api/transversales/{id}`
- **Controlador:** `TransversalController@eliminarTransversal`
- **Descripción:** Elimina una transversal existente.
- **Parámetros de URL:**
 - `id` (Obligatorio) → ID de la transversal
- **Respuestas:**
 - **200:** JSON confirmando la eliminación de la transversal.
 - **404:** Transversal no encontrada.

TutorController

1. Obtener grados de un tutor

- **Ruta:** `GET /api/tutores/{id}/grados`
 - **Controlador:** `TutorController@grados`
 - **Descripción:** Devuelve todos los grados asignados a un tutor específico.
 - **Parámetros de URL:**
 - `id` (Obligatorio) → ID del tutor
 - **Respuestas:**
 - **200:** JSON con los grados asignados al tutor.
 - **404:** Tutor no encontrado.
-

2. Obtener tutores disponibles

- **Ruta:** `GET /api/tutores/disponibles`
- **Controlador:** `TutorController@getTutoresDisponibles`
- **Descripción:** Devuelve una lista de tutores que actualmente no tienen ningún grado asignado.
- **Respuestas:**
 - **200:** JSON con los tutores libres, incluyendo:
 - `id` → ID del tutor
 - `nombre` → Nombre del tutor
 - `apellidos` → Apellidos del tutor
 - `email` → Correo electrónico del tutor
 - `nombre_completo` → Nombre completo (nombre + apellidos)

AlumnoImportController

1. Importar alumnos desde Excel

- **Ruta:** `POST /api/alumnos/importar`
 - **Controlador:** `AlumnoImportController@importar`
 - **Descripción:** Permite importar alumnos desde un archivo Excel (`.xls/x`).
 - **Campos del Request:**
 - `archivo` (Obligatorio) → Archivo Excel con los alumnos
 - `opciones` (Opcional) → JSON con opciones:
 - `crearGrados` → Booleano, si se deben crear (por defecto `true`)
 - `contraseñaDefecto` → Para alumnos importados (por defecto `"Egibide2025"`)
 - **Respuestas:**
 - **200:** JSON con el resultado de la importación:
 - `exito` → `true/false`
 - `importados` → número de alumnos importados
 - `errores` → array con errores por fila si los hay
 - **422:** Error de validación del archivo (tipo o tamaño)
 - **500:** Error al procesar el archivo
-

2. Vista previa de archivo Excel

- **Ruta:** `POST /api/alumnos/importar/vistaprevia`
- **Controlador:** `AlumnoImportController@vistaPrevia`
- **Descripción:** Obtiene una vista previa de las primeras 10 filas del Excel antes de importarlo.
- **Campos del Request:**
 - `archivo` (Obligatorio) → Archivo Excel (`.xlsx` o `.xls`)
- **Respuestas:**
 - **200:** JSON con:
 - `headers` → Array con los encabezados del Excel
 - `datos` → Primeras 10 filas de datos
 - `total` → Número total de filas en el archivo
 - **422:** Error de validación del archivo (tipo o tamaño)
 - **500:** Error al leer el archivo